

FAANG-Style System Design Summary: Invoice & PDF Generation System

End-to-End System Story

This system is a production-ready full-stack architecture where the frontend is responsible for UI and layout, while the backend owns rendering, business logic, and distribution. The React frontend is built using `npm run build` and served as static files by an Express backend. All APIs are exposed under `/api`, ensuring clean separation between UI routing and backend logic.

When a user creates an invoice, the frontend sends the complete invoice form data along with a unique slug to `/api/invoice/save`. The backend temporarily caches this data in memory with a TTL. This avoids unnecessary database writes and supports fast PDF generation.

For PDF generation, the frontend opens `/api/invoice/pdf/:slug`. The backend uses Puppeteer to load the actual frontend invoice page, waits for React rendering and fonts, and generates a pixel-perfect PDF. The generated PDF is uploaded to Google Drive based on business rules such as store code, and simultaneously returned to the user for download.

Key Design Decisions

- Reused frontend UI for PDF generation to avoid duplicate templates - Used in-memory cache for ephemeral invoice data - Extracted PDF logic into reusable utilities - Used OAuth2 and streaming uploads for secure, scalable storage - Non-blocking uploads to improve user experience

FAANG-Style Follow-Up Questions & Answers

Q: Why not generate PDFs directly from backend templates? A: Using the frontend UI guarantees visual consistency and avoids maintaining duplicate templates. Q: Why cache invoice data instead of saving directly to DB? A: PDF generation is transient; caching avoids unnecessary writes and improves performance. Q: How does this scale? A: Cache can be replaced with Redis, Puppeteer can be containerized, and Drive uploads can be async jobs. Q: What happens if the server crashes? A: For production, cache would be externalized (Redis) and PDFs regenerated on demand. Q: How do you secure Drive uploads? A: OAuth2 with refresh tokens stored in environment variables.

60-Second Interview Explanation

I built an invoice system where the frontend owns layout and the backend owns rendering and distribution. Invoice data is cached temporarily, Puppeteer renders the actual frontend page into a PDF, and the PDF is uploaded to Google Drive based on business rules and returned to the user.

instantly. This avoids duplicate templates, improves performance, and scales cleanly.